

Space Race Analysis

Deep Dive: architecture, code, and data, explained from first principles

Generated explainer for Salman Adnan's portfolio

Contents

1	What this project is	3
2	Directory layout	4
3	Core concepts, defined before first use	4
4	The data: mission_launches.csv	5
4.1	Schema	5
4.2	Why there are two junk index columns	5
4.3	Cleaning and derivation pipeline	6
4.4	Why the date parsing needs <code>format='mixed'</code>	7
4.5	Country lookup: how it works and where it fails	7
5	Part 1: the static analysis script (space_race_analysis.py)	7
5.1	Overview	7
5.2	Helper functions	7
5.3	The 17 charts, in the order they're produced	8
5.4	The spend-by-organisation table	9
6	Part 2: the interactive dashboard	9
6.1	Why a dashboard, and how it differs from the static script	9
6.2	Component map	10
6.3	<code>data_loader.py</code> in detail	10
6.4	<code>app.py</code> in detail	10
6.4.1	Startup (module-level code, runs once)	10
6.4.2	Filtering logic	11
6.4.3	Figure-building functions	11
6.4.4	Statistics	11
6.4.5	Serialising figures to JSON	12
6.4.6	Routes	12
6.4.7	Server startup	12
6.5	<code>templates/index.html</code> in detail	12
6.5.1	Structure	12
6.5.2	Client-side JavaScript	13
6.6	Sequence diagram: loading the dashboard, then applying a filter	14

7	Setup and running instructions	14
8	Known limitations (documented in the project's own README)	14
9	Provenance note	15

1 What this project is

This project answers a simple question with data: **who has launched the most rockets, what does a launch cost, and how did the USA and the USSR compare during the Cold War?** It does this by taking a raw, messy spreadsheet of every recorded space mission launch from 1957 (the start of the Space Race, when the Soviet Union launched Sputnik 1) through 2020 — 4,324 individual launch records from 56 different launching organisations — and turning it into charts and numbers a person can actually read.

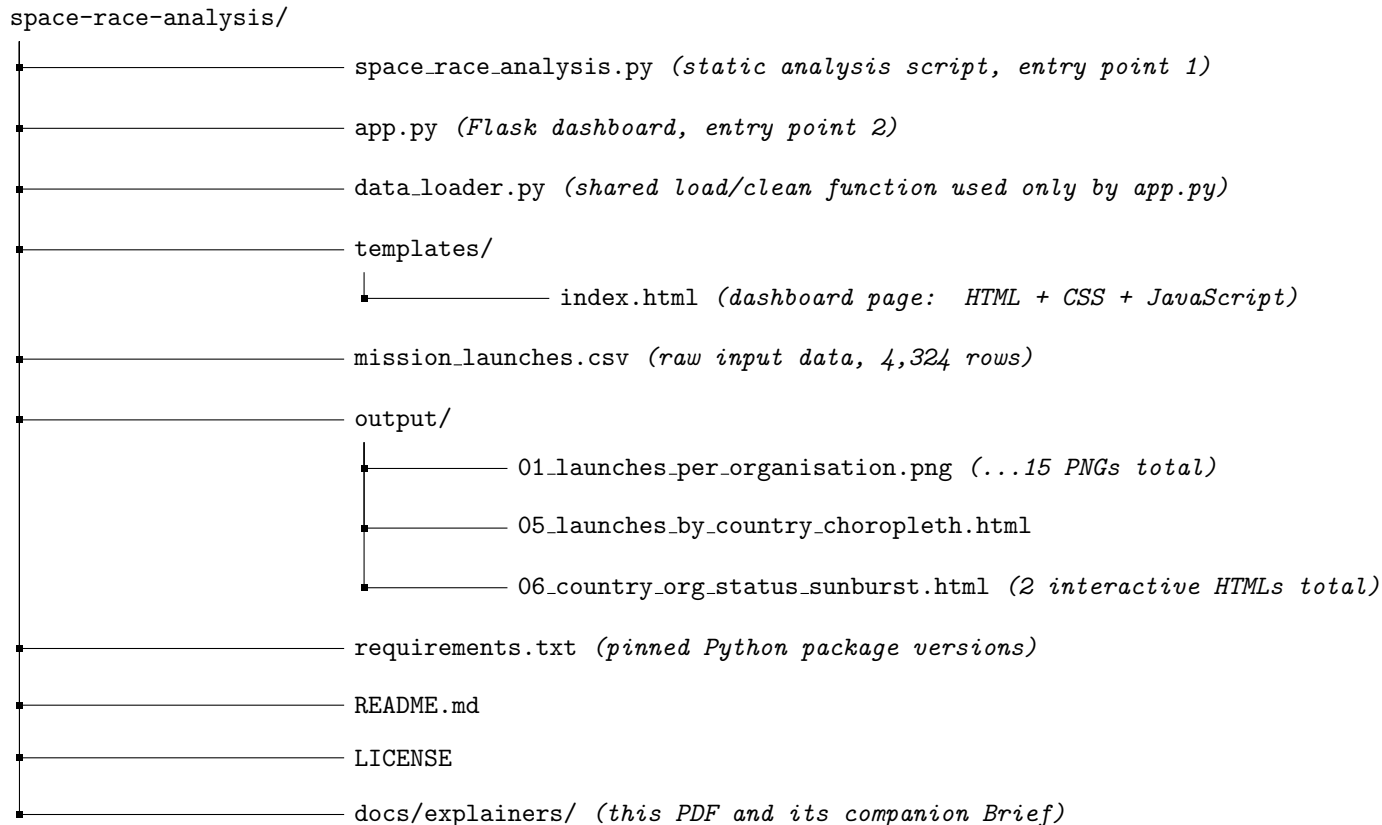
A **CSV file** (Comma-Separated Values: a plain-text table where each line is a row and commas separate the columns, openable in a spreadsheet program or read directly by code) called `mission_launches.csv` is the raw input. It was originally scraped (automatically extracted from a website's pages) from `nextspaceflight.com`, a site that tracks launch schedules and history.

The project ships in two parts that both read the same underlying data:

1. **A static analysis script**, `space_race_analysis.py`: run it once from a terminal and it writes 17 chart files to disk (as image files and small interactive web pages) and prints summary tables to the screen. This is the original, complete analysis.
2. **An interactive web dashboard**, `app.py` plus `templates/index.html`: a small web server (built with **Flask**, a lightweight Python framework for building web applications) that serves the same data through a page with filters — pick organisations, countries, and a year range, and five charts update live in the browser without reloading the page.

Both parts read the same CSV and apply the same cleaning rules, so their numbers always agree; the dashboard does not maintain a separate or precomputed copy of the analysis.

2 Directory layout



3 Core concepts, defined before first use

Python A general-purpose programming language. Both scripts here are Python programs, run with the command `python`.

pandas A Python library for working with tabular data. Its central object is the **DataFrame**: an in-memory table with named columns, similar to a spreadsheet, that supports filtering, grouping, and aggregation with code instead of mouse clicks.

matplotlib A Python library for drawing static charts (bar charts, histograms, line charts, pie charts) and saving them as image files (PNG).

Plotly A Python library (with a companion JavaScript library, Plotly.js) for drawing **interactive** charts: charts that support hovering for details, zooming, and clicking, either saved as a standalone HTML file or embedded in a web page.

Flask A lightweight Python **web framework**: a library that handles receiving HTTP requests from a browser and sending back responses (HTML pages or JSON data), so the author only has to write the logic for each URL (called a **route**).

JSON JavaScript Object Notation: a text format for structured data (numbers, strings, lists, nested objects) that both Python and JavaScript can read and write, used here to send filter results from the Flask server to the browser.

REST-style JSON API A URL that a client (here, the browser’s JavaScript) can send a request to and get structured data back, rather than a full HTML page. `/api/filter` in this project is one.

ISO alpha-3 country code A standardized three-letter code for a country (e.g. `USA`, `RUS`) defined by the International Organization for Standardization. Using a fixed vocabulary of codes instead of free-text country names is what lets the choropleth map look countries up reliably.

Choropleth map A map where each region (here, each country) is shaded by a numeric value; darker or more saturated shading means a higher value (here, more launches).

Sunburst chart A circular chart showing hierarchical (nested) data as concentric rings: the innermost ring is the top level of the hierarchy (here, Country), the next ring subdivides each of those into the next level (Organisation), and so on. Clicking a segment can “zoom in” to that branch.

NaN / null / missing value A placeholder pandas uses for “no value recorded” in a cell (e.g. a launch whose price was never disclosed).

Duplicate row Two or more rows that are identical across every column; typically an artifact of how data was collected, not real repeated events.

Virtual environment (venv) An isolated folder holding a project’s own copy of Python packages, so installing this project’s dependencies does not affect other Python projects on the same machine.

4 The data: `mission_launches.csv`

4.1 Schema

Column	Type	Meaning
(blank header)	int	Row index left over from the scrape; becomes a junk column.
<code>Unnamed: 0</code>	int	A second row-index column, same problem (see below).
<code>Organisation</code>	text	The launching entity, e.g. <code>SpaceX</code> , <code>NASA</code> , <code>Roscosmos</code> .
<code>Location</code>	text	Free-text launch site, e.g. <code>"LC-39A, Kennedy Space Center, Florida, USA"</code> .
<code>Date</code>	text	Launch date/time, inconsistently formatted across rows.
<code>Detail</code>	text	Rocket and payload description.
<code>Rocket_Status</code>	text	<code>StatusActive</code> or <code>StatusRetired</code> .
<code>Price</code>	number	Reported launch cost in millions of USD; present for only 964 of 4,324 rows ($\approx 22\%$), NaN elsewhere.
<code>Mission_Status</code>	text	<code>Success</code> , <code>Failure</code> , <code>Partial Failure</code> , or <code>Prelaunch Failure</code> .

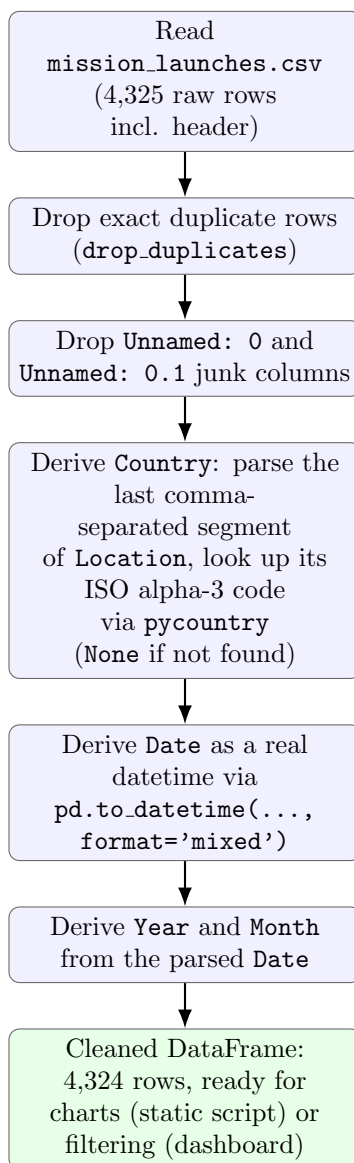
4.2 Why there are two junk index columns

The raw CSV’s header row has a blank first field followed by a field literally named `Unnamed: 0`. When pandas reads a CSV whose first column header is blank, it auto-names that column `Unnamed: 0` internally. That collides with the file’s own column that is already named `Unnamed: 0` in text, so pandas resolves the naming conflict by appending `.1` to the second occurrence. The result: the DataFrame ends up with columns `Unnamed: 0` and `Unnamed: 0.1`, both of which are just

row-number artifacts from however the original scrape was saved to CSV, carrying no analytical value. Both scripts drop them in the very first cleaning step.

4.3 Cleaning and derivation pipeline

Both `space_race_analysis.py` (inline) and `data_loader.py` (`load_data()`, used by the dashboard) perform the identical sequence below, which is why the dashboard's numbers always match the static script's:



`space_race_analysis.py` additionally prints exploratory statistics (`.info()`, `.isna().sum()`, `.duplicated().sum()`, `.describe()`) *before* the cleaning steps run, purely for the console output a human reads when running the script; `data_loader.py` intentionally skips these prints since the dashboard only needs the final cleaned frame, not a console narration.

4.4 Why the date parsing needs `format='mixed'`

The `Date` column mixes at least two textual formats across rows (some include a UTC time-zone offset, some don't; some include a day-of-week prefix like "Fri", some don't), because the data was scraped over time from a live website whose format apparently changed. Asking pandas to parse every row with one fixed format string fails on whichever rows do not match it. `format='mixed'` tells pandas to infer the format independently for each row instead of assuming the whole column follows one pattern; `utc=True` normalizes every parsed timestamp to UTC (Coordinated Universal Time, the global time-zone reference) so different original time zones compare correctly.

4.5 Country lookup: how it works and where it fails

```
def lookup_country(location):
    try:
        location = location.split(',')[1].strip()
        return pycountry.countries.lookup(location).alpha_3
    except LookupError:
        return None
```

This takes the text after the last comma in `Location` (e.g. "USA" from "LC-39A, Kennedy Space Center, Florida, USA") and asks the `pycountry` library (a package bundling the official ISO country-name/code registry) to resolve it to a three-letter code. `pycountry` only recognises *current* country names. Historical entities like the former USSR are not in that registry under any name the scraped location strings use, so `lookup` raises a `LookupError` for those rows, which the function catches and turns into `None` rather than letting the whole script crash. Every downstream chart that needs `Country` explicitly drops rows where it is `None` (`dropna(subset=['Country', ...])`), which means the choropleth and sunburst charts **undercount launches from historical USSR-era sites**; charts that don't depend on `Country` (e.g. launches per year, launches per organisation) are unaffected.

5 Part 1: the static analysis script (`space_race_analysis.py`)

5.1 Overview

This is a single linear script with one function, `main()`, called when the file is run directly (the `if __name__ == '__main__':` guard, a standard Python idiom meaning "only run this when the file is executed directly, not when it's imported by another file"). It has no classes and no configuration file; every step happens in order, top to bottom, matching what the script is for: a one-shot exploratory report rather than a long-running service.

5.2 Helper functions

`lookup_country(location)` Described above; identical logic to `data_loader.py`'s copy (the two files are independent on purpose — see Section 6).

`year(date) / month(date)` Small helpers that pull the calendar year (an integer) and month name (a string, e.g. "August", looked up from the `MONTHS` list by `date.month - 1` since Python lists are zero-indexed but calendar months start at 1) out of a parsed date.

`save_and_maybe_show_mpl(name)` Saves whatever matplotlib figure is currently "active" (matplotlib keeps one implicit current figure/axes unless told otherwise) to `output/{name}.png`, calls

`plt.show()` only if the environment variable `SHOW_PLOTS` is set to 1, then closes the figure to free memory, and prints the saved path.

`save_and_maybe_show_plotly(fig, name)` The Plotly equivalent: writes the figure to `output/{name}.html` via `fig.write_html()`, optionally calls `fig.show()` (opens a browser tab), and prints the saved path.

An **environment variable** is a named value set in the shell before running a program (e.g. `SHOW_PLOTS=1 python space_race_analysis.py`), read here via `os.environ.get('SHOW_PLOTS', '0') == '1'`, defaulting to off. This lets the same script run unattended in a terminal with no display (writing files only) or interactively with the charts popping up.

5.3 The 17 charts, in the order they're produced

#	Output file	Chart type	What it shows
1	01_launches_per_organisation.png	Bar	Launch count per organisation, all 56.
2	02_rocket_status.png	Bar	Count of <code>StatusActive</code> vs <code>StatusRetired</code> rockets.
3	03_mission_status.png	Bar	Count of each <code>MissionStatus</code> value.
4	04_price_histogram.png	Histogram	Distribution of launch <code>Price</code> , rows with no price dropped, x-axis clipped to \$0–1000M.
5	05_launches_by_country_colorblind.html	Choropleth (Plotly, interactive)	Launch count shaded per country.
6	06_country_org_status_subplots.html	Subplots (Plotly, interactive)	Country → Organisation → Mission Status, nested.
—	(console table, not a file)	—	Total and average spend per organisation, for the 25 organisations with nonzero launch count and nonzero total price (see Section 4.5).
7	07_launches_per_year.png	Bar	Launches per calendar year, every 3rd year labeled.
8	08_launches_per_month.png	Bar	Launches per calendar month, aggregated across all years.
9	09_average_price_over_time.png	Line	Mean <code>Price</code> per year (rows without a price dropped first).
10	10_top_10_organisations.png	Bar	The 10 organisations with the most launches.
11	11_top_10_organisations_multi-line.png	Multi-line	Yearly launch count for each of those top 10, one line per organisation.
12	12_usa_vs_ussr_over_time.png	Multi-line	Yearly launch count, NASA vs RVSN USSR only.
13	13_usa_vs_ussr_pie.png	Pie	Total-launch share, NASA vs RVSN USSR.
14	14_mission_failures_per_year.png	Bar	Count of <code>Failure</code> -status launches per year.
15	15_failure_rate_over_time.png	Line	% of launches with <code>Failure</code> status, per year (0–100 axis).

16	16_launches_per_year_sorted.png	Same data as #7 but with years sorted chronologically on the x-axis (chart #7's ordering follows <code>value_counts()</code> 's frequency order, not calendar order).
17	17_organisation_leader_by_year.png	Every organisation's launch count per year, one line each (the "who's ahead" view).

5.4 The spend-by-organisation table

```
money_spent_by_organizations = df_data.groupby('Organisation')['Price'].sum().reset_index()
money_spent_by_organizations['Total_Launches'] = money_spent_by_organizations['Organisation'].map(
    df_data['Organisation'].value_counts()
)
money_spent_by_organizations = money_spent_by_organizations[
    (money_spent_by_organizations['Total_Launches'] != 0) &
    (money_spent_by_organizations['Price'] != 0.00)
]
money_spent_by_organizations['Average_Price_Per_Launch'] = (
    money_spent_by_organizations['Price'] / money_spent_by_organizations['Total_Launches']
)
```

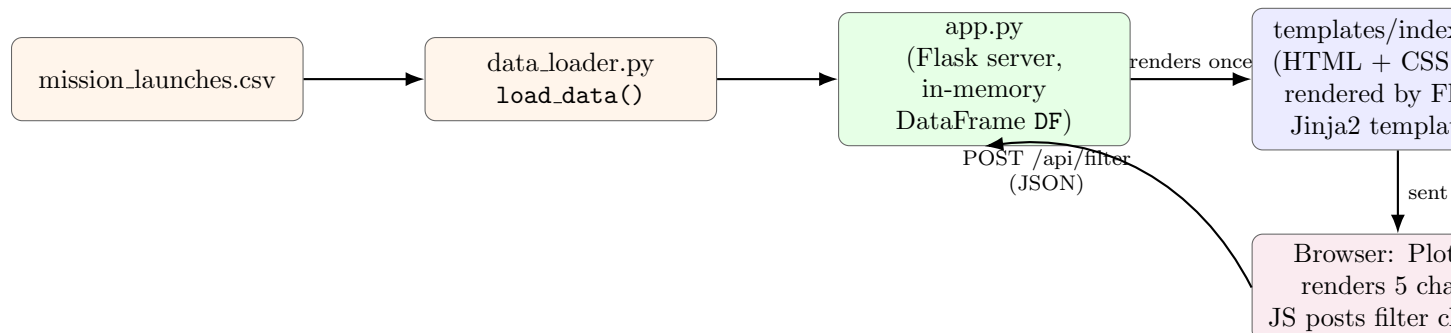
This groups all rows by `Organisation` and sums `Price` (NaNs are ignored by `.sum()` automatically), maps each organisation's *total* launch count (including launches with no reported price) onto that row, filters out organisations with zero total price (i.e. no organisation ever reports a price of exactly 0, this filter mainly excludes any that end up with a summed price of 0 because every one of their rows was NaN), and divides total price by total launches to get an average. This average is printed to the console, not charted. `groupby` (pandas' operation for splitting a DataFrame into groups sharing a column value, then computing something per group), `map` (looking up each row's new value from another Series by matching on a key, here the organisation name), and `.value_counts()` (pandas' function for counting how many times each distinct value appears in a column, returning a Series sorted by count descending) are the three pandas operations doing the work.

6 Part 2: the interactive dashboard

6.1 Why a dashboard, and how it differs from the static script

The static script produces a fixed snapshot: 17 files, generated once, that show every launch. The dashboard answers a different kind of question: "what did just NASA's and RVSN USSR's launches look like before 1992?", picked live, with real numbers computed on the spot rather than baked into a fixed image. It is additive, not a replacement: running `python space_race_analysis.py` still produces the same 17 files regardless of whether the dashboard exists.

6.2 Component map



Jinja2 is Flask’s built-in **templating engine**: a system for embedding placeholders (written `{{ ... }}` or `{% ... %}`) inside an HTML file that get replaced with real values (or loop over lists) when the server renders the page, which is how `index.html` receives the initial organisation list, country list, and first set of charts.

6.3 data_loader.py in detail

A single function, `load_data(csv_path=CSV_PATH)`, that performs the cleaning pipeline from Section 4.3 and returns the cleaned DataFrame. `CSV_PATH` is built from `os.path.abspath(os.path.dirname(__file__) + "/mission_launches.csv")`, i.e. the folder the script itself lives in, joined with the CSV’s filename — so the app finds the CSV correctly no matter what directory it’s launched from. Two more small functions live here: `lookup_country` (identical to the static script’s copy) and `country_name(alpha3)`, which does the reverse lookup (code "USA" → name "United States") for display in the dashboard’s country filter list, falling back to returning the raw code itself if `pycountry` can’t resolve it (guarding against an `AttributeError` when `pycountry.countries.get` returns `None` for an unrecognised code).

Deliberate duplication: `space_race_analysis.py` does not import from `data_loader.py`; each keeps its own copy of the cleaning logic. The README explains this is intentional — the static script is left untouched (it also prints exploratory NaN/duplicate counts *before* cleaning, which the shared loader intentionally skips since the dashboard doesn’t need that console narration).

6.4 app.py in detail

6.4.1 Startup (module-level code, runs once)

```

DF = load_data()
ALL_ORGANISATIONS = sorted(DF['Organisation'].dropna().unique().tolist())
ALL_COUNTRIES = sorted(
    ({'code': c, 'name': country_name(c)} for c in DF['Country'].dropna().unique()),
    key=lambda x: x['name'],
)
YEAR_MIN = int(DF['Year'].min())
YEAR_MAX = int(DF['Year'].max())
TOP_10_ORGANISATIONS = DF['Organisation'].value_counts().head(10).index.tolist()
  
```

The cleaned data is loaded exactly once, when the Flask process starts, and kept in a module-level variable `DF`. Every request re-slices this same in-memory DataFrame; nothing is reloaded from disk per request. `ALL_ORGANISATIONS` and `ALL_COUNTRIES` populate the two multi-select filter boxes;

`YEAR_MIN/ YEAR_MAX` bound the year inputs; `TOP_10_ORGANISATIONS` is computed once here (real counts, not hardcoded) and reused by the "Top 10 organisations" preset button on the client.

`STATUS_ORDER` and `STATUS_COLORS` fix a consistent left-to-right order and a consistent colour (green for Success, red for Failure, orange for Partial Failure, grey for Prelaunch Failure) for the mission-status bar chart, so the same status always gets the same colour regardless of which statuses are present in a given filtered slice.

6.4.2 Filtering logic

```
def apply_filters(organisations, countries, year_start, year_end):
    filtered = DF[(DF['Year'] >= year_start) & (DF['Year'] <= year_end)]
    if organisations:
        filtered = filtered[filtered['Organisation'].isin(organisations)]
    if countries:
        filtered = filtered[filtered['Country'].isin(countries)]
    return filtered
```

The year range always applies. The organisation and country filters only apply if the corresponding list is non-empty — an empty selection in the UI means "no restriction", not "match nothing", matching the UI copy ("none selected = all"). `.isin(...)` is pandas' membership test: keep rows whose value in that column is one of the given list.

6.4.3 Figure-building functions

Five functions each take the currently filtered DataFrame and return a Plotly figure object:

build_org_bar_figure Top 15 organisations by count in the current selection, blue bars.

build_year_line_figure Launch count per year in the current selection, sorted chronologically, purple line with markers.

build_status_figure Mission-status counts in the fixed order/colour scheme described above, restricted to whichever statuses are actually present.

build_choropleth_figure Country-level launch counts as a world map; if the filtered slice has zero rows with a known country, returns an empty figure with an explanatory title instead of erroring.

build_sunburst_figure Country → Organisation → Mission Status nesting for the current selection; same empty-figure guard as the choropleth.

Each of these mirrors a corresponding static chart from Section 5.3, rebuilt with Plotly (interactive) instead of matplotlib (static image) and parameterised on whatever subset of rows is currently selected instead of always using the full dataset.

6.4.4 Statistics

build_stats(filtered) returns a small dictionary summarising the current selection: total launch count, distinct organisation count, success rate (percentage of rows with `Mission.Status == 'Success'`), the top organisation by count and its count, and the min-max year range actually present, formatted as a string like "1957-1991". If the filtered slice is empty, it returns zeros/Nones instead of dividing by zero or indexing into an empty result.

6.4.5 Serialising figures to JSON

```
def figures_for(filtered):
    return {
        'org_bar': json.loads(json.dumps(build_org_bar_figure(filtered), cls=plotly.utils
        .PlotlyJSONEncoder)),
        ...
    }
```

A Plotly figure object is a Python object, not directly JSON; `plotly.utils.PlotlyJSONEncoder` is a custom JSON encoder (a plug-in that `json.dumps` consults for values it doesn't know how to serialise by default, such as numpy numeric types that appear inside Plotly figures) that knows how to turn a figure's internal data and layout attributes into plain JSON-compatible structures. The `json.dumps(...)` then `json.loads(...)` round-trip converts the figure to a JSON string and immediately back into a plain Python dictionary, which Flask's `jsonify` (for `/api/filter`) or Jinja2's template rendering (for `/`) can then embed directly.

6.4.6 Routes

GET `/` (`index()`) — builds figures and stats for the *full, unfiltered* dataset and renders `templates/index.html`, passing it the organisation list, country list, year bounds, the initial figures (as a JSON string embedded directly into the page), the initial stats, and the top-10 organisation list (also as embedded JSON, used by the "Top 10" preset button).

POST `/api/filter` (`api_filter()`) — reads a JSON request body of the shape `{"organisations": [...], "countries": [...], "year_start": 1957, "year_end": 2020}`, defaults `organisations/countries` to empty lists and `year_start/year_end` to the dataset's real min/max if absent, returns an HTTP 400 (Bad Request) with an error message if the year values aren't valid integers, otherwise calls `apply_filters` and returns fresh figures and stats as JSON via Flask's `jsonify`. Every response recomputes from the real dataframe; there is no cache of precomputed filter combinations.

6.4.7 Server startup

```
if __name__ == '__main__':
    app.run(debug=False, host='127.0.0.1', port=5000)
```

`host='127.0.0.1'` (also known as *localhost*) means the server only accepts connections from the same machine, not the network; `debug=False` turns off Flask's development-only debugger and auto-reload (safer defaults, though slower to iterate on than `debug=True` during development). Visiting <http://127.0.0.1:5000/> in a browser loads the dashboard.

6.5 templates/index.html in detail

6.5.1 Structure

The page has three stacked "cards" (visually boxed sections, styled via a small inline CSS stylesheet that also defines a light and dark colour scheme switched automatically via the CSS `prefers-color-scheme` media query, matching the visitor's OS theme):

1. **Filters card:** two multi-select boxes (organisations, countries, populated from the server-rendered lists) and two number inputs (year start/end), plus four buttons: Apply filters, Reset, "USA vs USSR (1957–1991)", and "Top 10 organisations".
2. **Stats card:** five stat tiles (Launches, Organisations, Success rate, Top organisation, Year range) that update after every filter change.
3. **Charts grid:** four charts in a 2-column responsive grid (organisation bar, year line, status bar, choropleth) plus the sunburst spanning the full width below them.

6.5.2 Client-side JavaScript

Initial render `const initialFigures = {{ figures_json | safe }}` embeds the server-computed figures directly into the page as a JavaScript object literal (the `| safe` Jinja2 filter tells Flask not to HTML-escape this JSON, since it needs to remain valid JavaScript). `Plotly.newPlot(...)` is called once per chart on page load using this embedded data — so the very first render needs no network request back to the server.

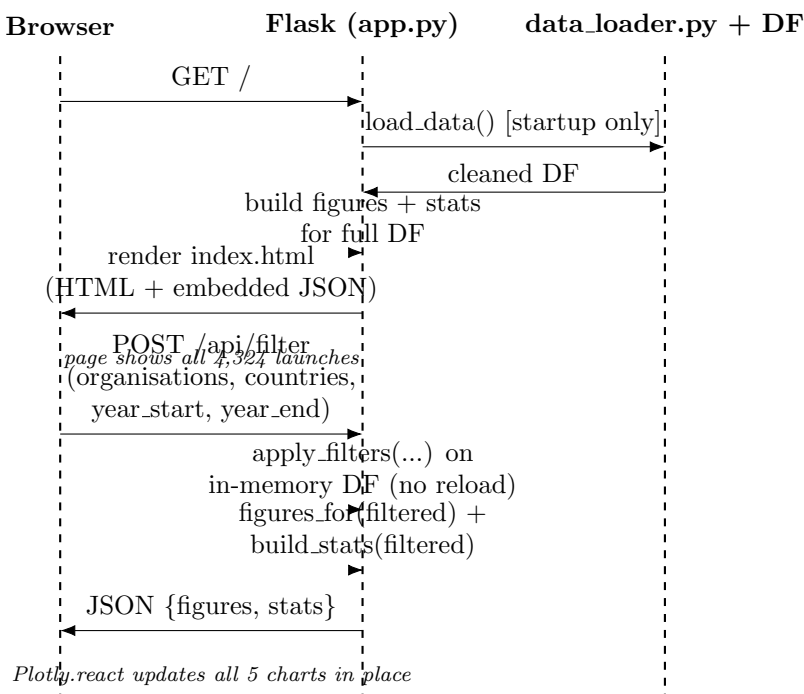
`selectedValues(selectEl) / setSelected(selectEl, values)` Helpers to read which options are currently highlighted in a multi-select box, and to programmatically highlight a given set of options (used by the preset buttons).

`updateStats(stats)` Writes the five stat values into the page after a filter change, showing "-" for any null value (e.g. no rows matched).

`applyFilters()` Reads the current filter controls, POSTs them as JSON to `/api/filter` via the browser's `fetch` API, and on success calls `Plotly.react(...)` for each of the five charts. `Plotly.react` is Plotly.js's "update this existing chart in place with new data" call, more efficient than tearing down and recreating the whole chart with `newPlot` again. On failure (non-OK HTTP response) it shows an inline error message instead of updating the charts with stale or missing data.

Buttons `Apply filters` calls `applyFilters()` directly. `Reset` clears both multi-selects and restores the full year range, then calls `applyFilters()`. The two preset buttons pre-select `['NASA', 'RVSN USSR']` with years capped at 1991, or the embedded `top100organisations` list with the full year range, respectively, then also call `applyFilters()`.

6.6 Sequence diagram: loading the dashboard, then applying a filter



7 Setup and running instructions

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Static analysis: writes 17 files to output/
python space_race_analysis.py
# ...or, to also pop up each chart interactively:
SHOW_PLOTS=1 python space_race_analysis.py

# Interactive dashboard: opens a local web server
python app.py
# then visit http://127.0.0.1:5000/ in a browser
```

Pinned dependencies (`requirements.txt`): `pandas` (data tables), `matplotlib` (static charts), `plotly` (interactive charts), `pycountry` (ISO country lookups), `Flask` (the dashboard's web server). No environment variables or secrets are required; `SHOW_PLOTS` is an optional toggle, not a credential.

8 Known limitations (documented in the project's own README)

- **Country undercounting.** As explained in Section 4.6, historical (pre-1991 USSR) launch sites cannot be resolved to a current ISO country by `pycountry`, so the choropleth and sunburst charts silently drop those rows. Year- and organisation-level charts are unaffected.
- **Sparse Price data.** Only about 22% of launches report a price; every price-based chart or table drops missing rows rather than treating them as zero, which would otherwise understate

average prices.

- **No price filter in the dashboard.** The dashboard covers organisation, country, and year, but not price, even though the static script's price histogram and spend table are part of the full analysis.
- **One long main() in the static script.** There is no way to regenerate a single chart without rerunning the whole script, and no unit-testable boundary around the cleaning step in isolation (acknowledged directly in the README's "What I'd do differently").

9 Provenance note

The README states this project began from a course exercise notebook template with the analysis code cells left blank (`Space_Missions_Analysis_(start).ipynb` in the original course material), and that the completed analysis was written as this standalone script instead of filling in the notebook; the notebook itself is not included in this repository since it carries no completed work.